

编译原理-中间代码生成 02

声明、表达式翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 5 月 15 日



福州大学
FUZHOU UNIVERSITY 1958



① 知识点回顾

② 中间代码生成策略

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

② 中间代码生成策略

① 知识点回顾

编译程序的阶段划分

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

② 中间代码生成策略

The logo of Fuzhou University is a large, faint, circular watermark in the background. It contains the university's name in English, 'FUZHOU UNIVERSITY', and in Chinese, '福州大学', along with the founding year '1958'.

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理, “译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;
- ④ 链接阶段 (Linking): 将一个或多个二进制目标文件进行关联, 处理外部函数跳转地址, 全局变量初始化等以生成可执行文件。

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理, “译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;
- ④ 链接阶段 (Linking): 将一个或多个二进制目标文件进行关联, 处理外部函数跳转地址, 全局变量初始化等以生成可执行文件。

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- ② 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- ③ 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

- 1 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- 2 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- 3 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

de Generation)：将抽象语法树转换为中间代码。中间代码是一种与硬件无关的低级表示形式，用于目标代码生成。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

Code Generation): 将抽象语法树转换为中间代码。中间代码是一种与硬件无关的低级表示形式,用于代码生成和目标代码生成。

中间代码进行优化,提高程序的运行效率。

式消除、循环优化等。

) 和后端优化 (与硬件相关)。

- 过程：常量折叠、公共子表达式消除、循环优化等。
- 分类：前端优化（与语言相关）和后端优化（与硬件相关）。
- 输出：优化后的中间代码。

- ④ 中间代码生成 (Intermediate Code Generation): 将抽象语法树转换为中间代码 (Intermediate Representation, IR)。中间代码是一种与硬件无关的低级表示形式 (如三地址码), 以便于后续优化和目标代码生成。
- ⑤ 优化阶段 (Optimization): 对中间代码进行优化, 提高程序的运行效率。
 - 过程: 常量折叠、公共子表达式消除、循环优化等。
 - 分类: 前端优化 (与语言相关) 和后端优化 (与硬件相关)。
 - 输出: 优化后的中间代码。
- ⑥ 目标代码生成 (Code Generation): 将优化后的中间代码转换为目标代码 (机器语言)。
 - 过程: 处理寄存器分配、指令选择等。
 - 输出: 目标代码 (如汇编、Java 字节码、Python 字节码等)。

- 4 中间代码生成 (Intermediate Code Generation): 将抽象语法树转换为中间代码 (Intermediate Representation, IR)。中间代码是一种与硬件无关的低级表示形式 (如三地址码), 以便于后续优化和目标代码生成。
- 5 优化阶段 (Optimization): 对中间代码进行优化, 提高程序的运行效率。
 - 过程: 常量折叠、公共子表达式消除、循环优化等。
 - 分类: 前端优化 (与语言相关) 和后端优化 (与硬件相关)。
 - 输出: 优化后的中间代码。
- 6 目标代码生成 (Code Generation): 将优化后的中间代码转换为目标代码 (机器语言)。
 - 过程: 处理寄存器分配、指令选择等。
 - 输出: 目标代码 (如汇编、Java 字节码、Python 字节码等)。

② 中间代码生成策略

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

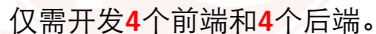
- 定义：中间代码是源程序在编译过程中的一种**中间表示（Intermediate Representation, IR）形式**。
- 特征：中间代码介于**高级语言**与**机器码**之间，具有**结构简单**、**机器无关性强**的特征。
- 目的：引入中间代码的核心目的是增强编译器的**可移植性**，并为**代码优化**提供结构化支持。

导致直接生成机器码难以跨平台运行，中间
标架构。

无关的优化操作（如常量折叠、公共子表达式
。

语法 / 语义分析）与后端（目标代码生成），

- **可移植性**：不同 CPU 指令集差异导致直接生成机器码难以跨平台运行，中间代码作为统一接口可适配多种目标架构。
- **代码优化**：中间代码支持与机器无关的优化操作（如常量折叠、公共子表达式消除），提升生成目标代码的效率。
- **开发维护**：分离编译器的前端（语法 / 语义分析）与后端（目标代码生成），降低模块耦合度，简化开发流程。



① 知识点回顾

编译程序的阶段划分

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

② 中间代码生成策略

按照功能与层次的分类

按照 IR 的层次进行划分有：

- 高层次 IR (HIR)：能够准确表达源语言的语义，用于发现语法错误、分析符号之间的依赖关系（以便进行跳转、判断方法的重载等）、根据需要自动生成或修改一些代码（提供重构能力）。
- 中间层次 IR (MIR)：大量的优化算法是可以通用的，没有必要依赖源语言的语法和语义，也没有必要依赖具体的 CPU 架构。这些优化包括部分算术优化、常量和变量传播、死代码删除等，实现这些类分析和优化功能的 IR 可以叫做 Middle IR, 简称 MIR.
- 低层次 IR (LIR)：它的指令通常可以与机器指令一一对应，比较容易翻译成机器指令 (或汇编代码)。因为 LIR 体现了 CPU 架构的底层特征，因此可以做一些于具体 CPU 架构相关的优化。

```
float a[10][20];  
a[i][j+2];
```

```
t1=a[i,j+2];
```

```
t1=j+2;  
t2=i*20;  
t3=t1+t2;  
t4=4*t3;  
t5=addr a;  
t6=t5+t4;  
t7=*t6;
```

```
r1=[fp-4];
r2=r1+2;
r3=[fp-8];
r4=r3*20;
r5=r4+t2;
r6=4*r5;
r7=fp-816;
f1=[r7+r6];
```

- 逆波兰式（后缀式）
- 图结构表示
- 三地址码



① 知识点回顾

编译程序的阶段划分

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

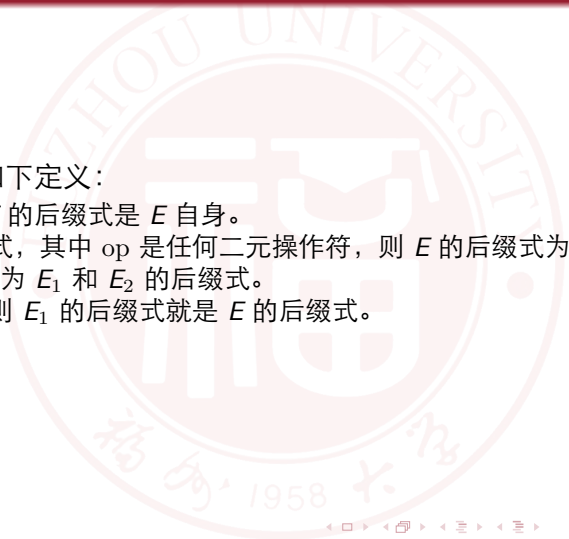
② 中间代码生成策略

逆波兰式（后缀式）

- 逆波兰表达式（Reverse Polish Notation, RPN）是一种数学表达式的书写形式。
- 它由波兰数学家扬·武卡谢维奇于 1920 年提出。
- 核心特征为运算符位于所有相关操作数之后，例如表达式“ $3+4$ ”的逆波兰式为“ $3\ 4\ +$ ”。
- 该表示法完全消除了括号需求，且运算符的优先级由操作符位置隐式确定，使得表达式解析和计算过程高度简化。

后缀式的定义

- 一个表达式 E 的后缀形式可以如下定义：
 - 若 E 是一个变量或常量，则 E 的后缀式是 E 自身。
 - 若 E 是形如 $E_1 \text{ op } E_2$ 的表达式，其中 op 是任何二元操作符，则 E 的后缀式为 $E_1 E_2 \text{ op}$ ，其中 E_1 和 E_2 分别为 E_1 和 E_2 的后缀式。
 - 若 E 是形如 (E_1) 的表达式，则 E_1 的后缀式就是 E 的后缀式。



后缀式的生成过程

将中缀表达式转换为逆波兰式的算法遵循以下步骤：

- 操作数处理：遇到数字或变量直接输出至结果序列。
- 运算符与栈操作：
 - 左括号'('：直接压入操作符栈。
 - 右括号')'：弹出栈顶元素至结果序列，直至遇到左括号（左括号弹出但不输出）。
 - 优先级比较：若当前运算符优先级低于或等于栈顶运算符，则弹出栈顶元素至结果序列，直至栈顶运算符优先级更低或遇到左括号，再将当前运算符入栈。
- 表达式扫描完成：将栈中剩余运算符依次弹出至结果序列。

- **人工可读性不足**：后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。
- **转换过程的计算开销**：对于大规模表达式或高频转换场景（如动态生成代码的场景），可能引发额外的性能损耗。在资源受限的嵌入式系统中仍需权衡效率。
- **特定场景的结构局限性**：在单线程顺序计算中表现高效，但对并行计算或硬件级优化的支持较弱。现代处理器常利用指令级并行技术（ILP）对独立运算单元加速，而逆波兰式的线性栈结构难以显式表达操作间的独立性，可能限制硬件优化潜力。对带有副作用（如赋值、函数调用）的表达式处理时，需引入额外标记或中间状态，增加实现复杂度。
- **语法扩展的适配难度**：需同步扩展优先级规则和栈处理逻辑，这对动态类型语言或元编程场景的灵活性构成挑战。

- **人工可读性不足：**后缀式采用运算符后置的书写规则，其符号顺序与人类自然思维习惯存在显著差异。相较于中缀表达式，后缀式在代码调试或手写验证时需额外认知转换成本。
- **转换过程的计算开销：**对于大规模表达式或高频转换场景（如动态生成代码的场景），可能引发额外的性能损耗。在资源受限的嵌入式系统中仍需权衡效率。
- **特定场景的结构局限性：**在单线程顺序计算中表现高效，但对并行计算或硬件级优化的支持较弱。现代处理器常利用指令级并行技术（ILP）对独立运算单元加速，而逆波兰式的线性栈结构难以显式表达操作间的独立性，可能限制硬件优化潜力。对带有副作用（如赋值、函数调用）的表达式处理时，需引入额外标记或中间状态，增加实现复杂度。
- **语法扩展的适配难度：**需同步扩展优先级规则和栈处理逻辑，这对动态类型语言或元编程场景的灵活性构成挑战。



① 知识点回顾

编译程序的阶段划分
 中间代码生成阶段
 中间表示的分类
 后缀式
 树/图结构
 三地址码
 三地址码的表示与存储

② 中间代码生成策略

图结构 IR

图结构 IR 按照实际的图的类型有可以分为**树状中间表示 (Tree-based IR)** 和**图结构中间表示 (Graph-based IR)**。

树状中间表示 (Tree-based IR)

- 树状中间表示 (Tree-based IR) 是虎书¹所使用的一种中间表示方法。
- 树状中间表示以树形数据结构为基础，通常由抽象语法树 (AST) 演化而来。每个节点代表一个操作 (如算术运算、函数调用等)，子节点表示操作数或子表达式。
- 优劣势：
 - 层次化结构：严格遵循程序的语法结构，便于语法分析和语义检查。
 - 显式操作关系：通过父子节点关系直接体现操作符与操作数的绑定顺序。
 - 局部性优化：适合局部表达式优化 (如常量折叠)，但对跨作用域或复杂控制流的支持较弱。

¹Modern Compiler Implementation in Java/C++/ML, Second Edition, Andrew W. Appel & Jens Palsberg



据)和边(表示控制流或数据依赖)构建程序控制流图。图中,节点表示程序执行点,边表示程序执行路径的跳转。边表示数据依赖关系。

图 10-1-1 展示了通过 Phi 函数处理分支合并。

- 控制流图 (CFG): 基本块作为节点, 边表示程序执行路径的跳转。
- 数据流图 (DFG): 节点为操作, 边表示数据依赖关系。
- 静态单赋值 (SSA): 变量仅赋值一次, 通过 Phi 函数处理分支合并。

- 语法树或 DAG 图的结点通常存放在一个记录数组中
 - 数组的每一行是一个记录，表示一个结点；
 - 数组下标为对应结点的值编码；
 - 叶结点包含记号字段和一个附加字段，存放词法值（标识符时为指向符号表相应项目的指针，数字时是常量）；
 - 内部结点包含运算符字段和若干附加字段，指向其儿子结点；

1	id	0x123	
2	num	10	
3	+	1	2
4	=	1	3
	⋮		



1	id	0x123	
2	num	10	
3	+	1	2
4	=	1	3
	⋮		

树 IR 和图结构 IR 总结

- 现代编译器常采用混合策略：前端使用树状 IR 进行语法和语义分析，后端转换为图结构 IR 实现深度优化。例如，LLVM 在生成低级中间表示（LLVM IR）前，可能先构建控制流图以支持循环优化和指令调度。
- 树状 IR 与图结构 IR 分别服务于编译过程的不同需求。树状 IR 以简洁性和直观性见长，适合前端处理；图结构 IR 以强大的分析和优化能力为核心，是后端优化的基石。理解两者的设计哲学和应用边界，对编译器开发及中间表示的选择至关重要。

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

② 中间代码生成策略

三地址码

三地址码 (Triple Address Code, TAC) 是一种中间表示形式, 每条指令最多包含三个操作数, 其基本结构可分解为四元组形式:

(运算符, 操作数 1, 操作数 2, 结果)

例

表达式 $a = b + c * d$ 可转换为:

$t1 = c * d$

$a = b + t1$

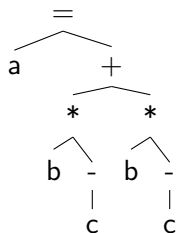
三地址码的设计目标: 通过引入临时变量分解复合表达式, 简化语法树或图结构的复杂性, 为后续优化和机器无关代码生成提供线性化、标准化的中间表示。

“三”是一个**泛指**, 具体由**四**元式、**三**元式、间接**三**元式等多种形式。

Pay Attention!!!

三地址码既可以是树形 IR（如 AST）的线性表示，也可以是 DAG 的线性表示。

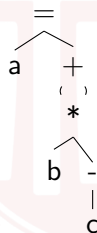
例



AST 的三地址码:

```

t1 = -c;
t2 = b * t1;
t3 = -c;
t4 = b * t3;
t5 = t2 + t4;
a = t5;
    
```



DAG 的三地址码:

```

t1 = -c;
t2 = b * t1;
t3 = t2 + t2;
a = t3;
    
```

op 是双目运算符;

内存单元的位置中存放的值赋给 x;

内存单元的位置中的内容置为 y 的值;

的值设置为 y 的地址;

y 的值;

x;

- op 是双目运算符;
- 内存单元的位置中存放的值赋给 x;
- 内存单元的位置中的内容置为 y 的值;
- 的值设置为 y 的地址;
- y 的值;
- x;

一步执行带有标号 L 的三地址指令；
 转至 L
 时跳转至 L。
 和 y 之间满足 relop 关系时跳转至 L；

- 一步执行带有标号 L 的三地址指令；
 转至 L
 时跳转至 L。
 和 y 之间满足 relop 关系时跳转至 L；

- 过程调用及返回：
 - 参数传递 param x: 传递参数 x;
 - 过程调用 call p, n: 调用 p 过程, 实参个数为 n;
 - y = call p, n: 函数调用——调用 p 过程, 返回值为 y;

过程调用 `heapsort(m, n)` 对应的三地址码为?

```
param m;
```

```
param n;
```

```
call heapsort, 2;
```

中间代码生成阶段

中间表示的分类

后缀式

树/图结构

三地址码

三地址码的表示与存储

② 中间代码生成策略

三地址码的表示方法之四元式

- 三地址码说明了各类指令的组成部分，但未描述指令在**某个数据结构中的表示**方法。它可以用**四元式**、**三元式**和**间接三元式**来描述。
- 四元式是一个带有 4 个字段的记录结构：op, arg1, arg2, result, 其中 arg1, arg2 和 result 通常指向符号表项目的入口。
- 特别地，有一些项目无法使用四元组表示，即四元式特例：
 - $x = op\ y$ 和 $x = y$ ：不使用 arg2
 - param 运算：不使用 arg2 和 result
 - 转移指令：将目标标号 (label) 存入 result

例 $a = b * (-c) + b * (-c)$ 的四元式

四元式表示：

AST 的三地址码：

$t1 = -c;$

$t2 = b * t1;$

$t3 = -c;$

$t4 = b * t3;$

$t5 = t2 + t4;$

$a = t5;$

序号	op	arg1	arg2	result
(0)	uminus	c	N/A	t1
(1)	*	b	t1	t2
(2)	uminus	c	N/A	t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5	N/A	a

三地址码之三元式

- 三元式是 3 个字段的记录结构：op, arg1, arg2;
- 三元式可以避免引入临时变量（如 t1, t2, t3）;
- 使用运算 $x \text{ op } y$ 的位置来引用它的运算结果;
- 用带有括号的数字表示指向三元式结构的指针;

例 $(a=b*(-c)+b*(-c))$ 的三元式)

四元式表示:

三元式表示法:

AST 的三地址码:

$$t_1 = -c;$$

```
t2 = b * t1;
```

```
t3 = -c;
```

```
t4 = b * t3;
```

```
t5 = t2 + t4;
```

```
a = t5;
```

序号	op	arg1	arg2	result
(0)	uminus	c	N/A	t1
(1)	*	b	t1	t2
(2)	uminus	c	N/A	t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	=	t5	N/A	a

序号	op	arg1	arg2
(0)	uminus	c	—
(1)	*	b	(0)
(2)	uminus	c	—
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

位置常常会发生变化。当移动一个变量的指令
改引用该指令地址的所有指令。

三元式表示法的问题:

- 在优化编译器中，指令实际存储位置常常会发生变化。当移动一个变量的指令时，使用三元式表示法便需要修改引用该指令地址的所有指令。

位置常常会发生变化。当移动一个变量的指令时，要修改引用该指令地址的所有指令。

- 间接码表；

指示器表，按运算的先后次序列出有关三元式在表中的位置。可以通过对该表的重新排序来移动指令的位置。

- 在优化编译器中，指令实际存储位置常常会发生变化。当移动一个变量的指令时，使用三元式表示法便需要修改引用该指令地址的所有指令。

- 间接三元式方案包括三元式表 + 间接码表；
- 间接码表（指针数组）：一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置，优化编译器可以通过对该表的重新排序来移动指令的位置。
- 优点：方便优化，节省空间

式)

	op	arg1	arg2
(0)	uminus	c	—
(1)	*	b	(0)
(2)	+	(1)	(1)
(3)	=	a	(2)

	op	arg1	arg2
(0)	uminus	c	—
(1)	*	b	(0)
(2)	+	(1)	(1)
(3)	=	a	(2)

例

构造下述语句的三地址码、四元式、三元式、间接三元式:

$$X = (A + B) * C;$$
$$Y = D \uparrow (A + B) ;$$

三地址码：

$$t1 = A + B;$$
$$X = t_1 * C;$$
$$Y = D \uparrow t1;$$

四元式表示法:

序号	op	arg1	arg2	result
(0)	+	A	B	t1
(1)	*	C	t1	X
(2)	↑	D	t1	Y

三元式表示法:

序号	op	arg1	arg2
(0)	+	A	B
(1)	*	C	(0)
(2)	=	X	(1)
(3)	↑	D	(0)
(4)	=	Y	(3)



① 知识点回顾

② 中间代码生成策略

声明处理

表达式翻译

控制结构处理

布尔表达式值的计算方法

析的抽象语法树或符号表。
址码、四元式或其他中间形式。

- 生成流程：
 - 输入：经过语法分析与语义分析的抽象语法树或符号表。
 - 输出：与目标机器无关的三地址码、四元式或其他中间形式。
 - 关键步骤：

中间代码生成策略

- 生成流程：
 - 输入：经过语法分析与语义分析的抽象语法树或符号表。
 - 输出：与目标机器无关的三地址码、四元式或其他中间形式。
 - 关键步骤：
 - 声明处理：将变量类型、作用域等信息登记至符号表，分配存储地址（如相对偏移量）。

析的抽象语法树或符号表。
址码、四元式或其他中间形式。

用域等信息登记至符号表，分配存储地址（如相对偏移
树，生成运算序列（如利用语法制导翻译生成四元式）。

- 生成流程：
 - 输入：经过语法分析与语义分析的抽象语法树或符号表。
 - 输出：与目标机器无关的三地址码、四元式或其他中间形式。
 - 关键步骤：
 - 声明处理：将变量类型、作用域等信息登记至符号表，分配存储地址（如相对偏移量）。
 - 表达式翻译：递归遍历语法树，生成运算序列（如利用语法制导翻译生成四元式）。

析的抽象语法树或符号表。
址码、四元式或其他中间形式。

用域等信息登记至符号表，分配存储地址（如相对偏移
树，生成运算序列（如利用语法制导翻译生成四元式）。
、循环语句生成跳转指令标签。

- 析的抽象语法树或符号表。
地址码、四元式或其他中间形式。
- 用域等信息登记至符号表，分配存储地址（如相对偏移量）。
- 树，生成运算序列（如利用语法制导翻译生成四元式）。
- 、循环语句生成跳转指令标签。

析的抽象语法树或符号表。

地址码、四元式或其他中间形式。

作用域等信息登记至符号表，分配存储地址（如相对偏移量）。

生成运算序列（如利用语法制导翻译生成四元式）。

为分支、循环语句生成跳转指令标签。

生成常量值。

生成有效代码段。

保存计算的中间结果。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ≡ ▶ ≡ ▶ ↺ 🔍 ↻



① 知识点回顾

② 中间代码生成策略

声明处理

表达式翻译

控制结构处理

布尔表达式值的计算方法

[illegible]

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。



类型及存储空间的布局问题。

空间是在运行时进行分配的。在编译时检查布局 (layout)。

- 类型及存储空间的布局问题。
- 空间是在运行时进行分配的。在编译时检查布局 (layout)。

类型及存储空间的布局问题。

空间是在**运行时**进行分配的。在**编译时**检查布局 (layout)。

相对地址是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

- 类型及存储空间的布局**问题。
- 空间是在**运行时**进行分配的。在**编译时**检查布局 (layout)。
- 相对地址**是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

类型及存储空间的布局问题。

空间是在**运行时**进行分配的。在**编译时**检查布局 (layout)。

相对地址是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：

类型及存储空间的布局问题。

空间是在**运行时**进行分配的。在**编译时**检查布局 (layout)。

相对地址是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

定程序在运行时的行为。

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。

[illegible]

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。

- 类型及存储空间的布局问题。
- 空间是在运行时进行分配的。在编译时检查布局 (layout)。
- 相对地址是指它相对于数据区域开始位置 (如第一个字节) 的偏移量。
- 定程序在运行时的行为。
- 运算符的预期类型相匹配。

- 类型及存储空间的布局问题。
- 空间是在运行时进行分配的。在编译时检查布局 (layout)。
- 相对地址是指它相对于数据区域开始位置 (如第一个字节) 的偏移量。
- 定程序在运行时的行为。
- 运算符的预期类型相匹配。
- ;

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - ① 确定一个名需要的存储空间；
 - ② 计算一个数组元素引用的地址；

类型及存储空间的布局问题。

空间是在运行时进行分配的。在编译时检查布局 (layout)。

相对地址是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

定程序在运行时的行为。

运算符的预期类型相匹配。

;

址;

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - ① 确定一个名需要的存储空间；
 - ② 计算一个数组元素引用的地址；
 - ③ 插入显式的类型转换；
 - ④ 选择算术运算符的正确版本；

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - 1 确定一个名需要的存储空间；
 - 2 计算一个数组元素引用的地址；
 - 3 插入显式的类型转换；
 - 4 选择算术运算符的正确版本；

- **类型**自身也有**结构**，用**类型表达式**来表示。



式来表示。

收、浮点、字符、布尔、无类型；

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

式来表示。

整、浮点、字符、布尔、无类型；

- 式来表示。
- 整、浮点、字符、布尔、无类型；

式来表示。

收、浮点、字符、布尔、无类型；

；

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type)：整数、浮点、字符、布尔、无类型；
- 类型表达式与类型别名：
 - 类型表达式用于说明一个类型；

式来表示。

文、浮点、字符、布尔、无类型；

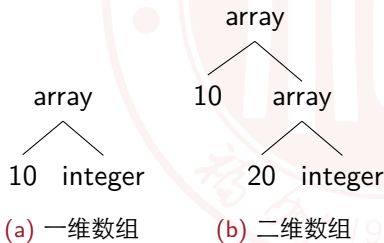
；

价；

- 式来表示。
- 文、浮点、字符、布尔、无类型；
- ；
- 价；

常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型（primitive type）：整数、浮点、字符、布尔、无类型；
- 类型表达式与类型别名：
 - 类型表达式用于说明一个类型；
 - 类型别名与一串类型表达式等价；
- 数组类型：一维数组、多维数组；如 `array(N, T)` 说明了一个有 10 个 T 类型元素构成的数组。



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型（primitive type）：整数、浮点、字符、布尔、无类型；
- 类型表达式与类型别名：
 - 类型表达式用于说明一个类型；
 - 类型别名与一串类型表达式等价；
- 数组类型：一维数组、多维数组；如 `array(N, T)` 说明了一个有 10 个 T 类型元素构成的数组类型。
- 指针类型：用 `pointer(T)` 表示指向数据类型 T 的指针类型。
- 笛卡尔算符 $\times$ ：如果 T_1 与 T_2 是合法的类型表达式，则 $T_1 \times T_2$ 也是合法的类型表达式。（用在哪里？）

函数 `int *f(char a, char b);` 如何表示?

类型表达式: $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{int})$

常见类型表达式

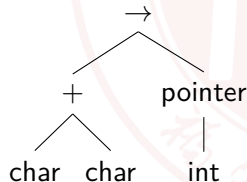
- 函数声明符 $\rightarrow$: 若 T_1 、 T_2 、...、 T_n 和 R 是类型表达式, 则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是合法的类型表达式, 用于表示一个函数。

例

函数 `int *f(char a, char b);` 如何表示?

类型表达式: $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{int})$

对应的树形表示为:



1. *Journal of the American Medical Association*, 2000; 283: 2689-2696.

法同由一列估计值和数列从当前估计值

```
struct person {
```

```
struct person table[50];
```

²结构体、类、命名元组 (namedtuple)

授课教师: 柯岩宇 福州大学计算机与大数据学院

任务是收集标识符的类型、所需存储单元大小、
一个相对地址，并将它们保存在相应的符号表中。
(type) 在运行时刻所需的存储单元数量称为类型
宽度为每一个名字分配一个相对地址 (offset)

- 对于声明语句，语义分析的主要任务是收集标识符的类型、所需存储单元大小等属性信息，并为每个名字分配一个相对地址，并将它们保存在相应的符号表记录中。
 - 从类型表达式可以知道该类型 (type) 在运行时刻所需的存储单元数量称为类型的宽度 (width)
 - 在编译时刻，可以使用类型的宽度为每一个名字分配一个相对地址 (offset)

- 翻译要求：确定每个变量的**类型**表达式和相对地址。

- 翻译要求：确定每个变量的**类型**表达式和**相对地址**。

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

- 翻译要求：确定每个变量的**类型**表达式和相对地址。
- SDD 如何设计？

- 翻译要求：确定每个变量的**类型**表达式和相对地址。
- SDD 如何设计？

- 翻译要求：确定每个变量的**类型**表达式和相对地址。
- SDD 如何设计？

- 翻译要求：确定每个变量的**类型**、**表达式**和**相对地址**。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width

- 1 $P \rightarrow D$
- 2 $D \rightarrow T \text{ id}; D$
- 3 $D \rightarrow$
- 4 $T \rightarrow BC$
- 5 $B \rightarrow \text{int}$
- 6 $B \rightarrow \text{real}$
- 7 $C \rightarrow$
- 8 $C \rightarrow [\text{num}]C1$

- 翻译要求：确定每个变量的**类型**表达式和**相对地址**。
- SDD 如何设计？
 - B, C, T 的综合属性 `type, width;`

- 翻译要求：确定每个变量的**类型**和**相对地址**。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width
- SDT 方案需要的辅助变量：

- 1 $P \rightarrow D$
- 2 $D \rightarrow T \text{ id}; D$
- 3 $D \rightarrow$
- 4 $T \rightarrow BC$
- 5 $B \rightarrow \text{int}$
- 6 $B \rightarrow \text{real}$
- 7 $C \rightarrow$
- 8 $C \rightarrow [\text{num}]C1$

- 翻译要求：确定每个变量的类型表达式和相对地址。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width；
- SDT 方案需要的辅助变量：

- 翻译要求：确定每个变量的**类型**、**表达式**和**相对地址**。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width
- SDT 方案需要的辅助变量：
 - offset：下一个可用的相对地址

- 1 $P \rightarrow D$
- 2 $D \rightarrow T \text{ id}; D$
- 3 $D \rightarrow$
- 4 $T \rightarrow BC$
- 5 $B \rightarrow \text{int}$
- 6 $B \rightarrow \text{real}$
- 7 $C \rightarrow$
- 8 $C \rightarrow [\text{num}]C1$

- 翻译要求：确定每个变量的**类型**表达式和**相对地址**。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width;
- SDT 方案需要的辅助变量：
 - offset：下一个可用的相对地址；

- 翻译要求：确定每个变量的达式和相对地址。
- SDD 如何设计？
 - B, C, T 的综合属性 type
- SDT 方案需要的辅助变量：
 - offset：下一个可用的相对地址
- 在考虑第一个声明之前，offset 设置为 0。每处理一个变量加入符号表，它的相对地址为 offset 的当前值，其宽度为 offset 上。

- 1 $P \rightarrow D$
- 2 $D \rightarrow T \text{ id}; D$
- 3 $D \rightarrow$
- 4 $T \rightarrow BC$
- 5 $B \rightarrow \text{int}$
- 6 $B \rightarrow \text{real}$
- 7 $C \rightarrow$
- 8 $C \rightarrow [\text{num}]C1$

- 翻译要求：确定每个变量的**类型**表达式和相对地址。
- SDD 如何设计？
 - B, C, T 的综合属性 type, width；
- SDT 方案需要的辅助变量：
 - offset：下一个可用的相对地址；
- 在考虑第一个声明之前，offset 被设置为 0。每处理一个变量 x ， x 被加入符号表，它的相对地址被设置为 offset 的当前值，其宽度被加到 offset 上。

- 类型表达式的生成需要将类型和度信息从语法分析树中的 B 结点递到对应于产生式 $C \rightarrow$ 的结点。可设置继承属性 inh 实现。为提高效率，可以设置临时变量 t 和 v 存 B 的 type 和 width 值。

- 类型表达式的生成需要将类型和宽度信息从语法分析树中的 B 结点传递到对应于产生式 $C \rightarrow$ 的结点，可设置继承属性 inh 实现。为提高效率，可以设置临时变量 t 和 w 暂存 B 的 type 和 width 值。

- 类型表达式的生成需要将类型和宽度信息从语法分析树中的 B 结点传递到对应于产生式 $C \rightarrow$ 的结点，可设置继承属性 inh 实现。为提高效率，可以设置临时变量 t 和 w 暂存 B 的 type 和 width 值。

变量声明语句的 SDT

- ① $P \rightarrow \{\text{offset} = 0;\} D$
- ② $D \rightarrow T \text{ id}; \{\text{st}[\text{top}].\text{put}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width}\} D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B \{ t = B.\text{type}; w = B.\text{width}; \} C \{ T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width}; \}$
- ⑤ $B \rightarrow \text{int} \{ B.\text{type} = \text{int}; B.\text{width} = 4; \}$
- ⑥ $B \rightarrow \text{real} \{ B.\text{type} = \text{real}; B.\text{width} = 8; \}$
- ⑦ $C \rightarrow \varepsilon \{ C.\text{type} = t; C.\text{width} = w; \}$
- ⑧ $C \rightarrow [\text{num}]C1 \{ C.\text{type} = \text{array}(\text{num.val}, C1.\text{type});$
 $C.\text{width} = \text{num.val} * C1.\text{width}; \}$



① 知识点回顾

② 中间代码生成策略

声明处理

表达式翻译

算术表达式

增量翻译

带数组引用的赋值语句的翻译

控制结构处理

布尔表达式值的计算方法

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

例

将 $x = (a + b) * c$ 翻译为

```
t1 = a + b;  
t2 = t1 * c;  
x = t2;
```

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

- 例

```
t1 = a + b;  
t2 = t1 * c;  
x = t2;
```

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

例

将 $x = (a + b) * c$ 翻译为

```
t1 = a + b;  
t2 = t1 * c;  
x = t2;
```

如何设计?

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

- 例

```
t1 = a + b;  
t2 = t1 * c;  
x = t2;
```

如何设计？

赋值语句 SDT 方案:

- 对于 S 符号:

对

赋值语句 SDT 方案:

- 1 $S \rightarrow id = E;$
- 2 $E \rightarrow E1 + E2$
- 3 $E \rightarrow E1 * E2$
- 4 $E \rightarrow -E1$
- 5 $E \rightarrow (E1)$
- 6 $E \rightarrow id$

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

赋值语句 SDT 方案:

- 对于 S 符号:
- 继承属性?

对

赋值语句 SDT 方案:

- 对于 S 符号:
 - 继承属性?

赋值语句翻译过程的主要任务：**生成对表达式求值的三地址码序列。**

赋值语句 SDT 方案:

- 对于 S 符号:
 - 继承属性?
 - 综合属性 code: 赋值语句 S 的地址代码。

赋值语句 SDT 方案:

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的三地址代码。

赋值语句 SDT 方案:

- 对于 S 符号:
 - 继承属性?
 - 综合属性 code: 赋值语句 S 的地址代码。
- 对于 E 符号:

- 1 $S \rightarrow id = E;$
- 2 $E \rightarrow E1 + E2$
- 3 $E \rightarrow E1 * E2$
- 4 $E \rightarrow -E1$
- 5 $E \rightarrow (E1)$
- 6 $E \rightarrow id$

赋值语句 SDT 方案:

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的地址代码。
- 对于 E 符号：

赋值语句 SDT 方案：

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的地址代码。
- 对于 E 符号：
 - 综合属性 code：对 E 求值的三址语句序列；

- 1 $S \rightarrow id = E;$
- 2 $E \rightarrow E1 + E2$
- 3 $E \rightarrow E1 * E2$
- 4 $E \rightarrow -E1$
- 5 $E \rightarrow (E1)$
- 6 $E \rightarrow id$

赋值语句 SDT 方案:

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的三地址代码。
- 对于 E 符号：
 - 综合属性 code：对 E 求值的三地址语句序列；

赋值语句 SDT 方案:

- 对于 S 符号:
 - 继承属性?
 - 综合属性 code: 赋值语句 S 的地址代码。
- 对于 E 符号:
 - 综合属性 code: 对 E 求值的三址语句序列;
 - 综合属性 addr: 存放 E 值的地址 (名字或临时变量);

- 1 $S \rightarrow id = E;$
- 2 $E \rightarrow E1 + E2$
- 3 $E \rightarrow E1 * E2$
- 4 $E \rightarrow -E1$
- 5 $E \rightarrow (E1)$
- 6 $E \rightarrow id$

赋值语句 SDT 方案:

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的三地址代码。
- 对于 E 符号：
 - 综合属性 code：对 E 求值的三地址语句序列；
 - 综合属性 addr：存放 E 值的地址（名字或临时变量）；

赋值语句 SDT 方案:

- 对于 S 符号:
 - 继承属性?
 - 综合属性 code: 赋值语句 S 的地址代码。
- 对于 E 符号:
 - 综合属性 code: 对 E 求值的地址语句序列;
 - 综合属性 addr: 存放 E 值的地址 (名字或临时变量);
- 全局辅助函数 newTemp(): 生成新的临时变量 t, 返回 t 的地址 (名字);

- 1 $S \rightarrow id = E;$
- 2 $E \rightarrow E1 + E2$
- 3 $E \rightarrow E1 * E2$
- 4 $E \rightarrow -E1$
- 5 $E \rightarrow (E1)$
- 6 $E \rightarrow id$

赋值语句 SDT 方案:

- 对于 S 符号：
 - 继承属性？
 - 综合属性 code：赋值语句 S 的三地址代码。
- 对于 E 符号：
 - 综合属性 code：对 E 求值的三地址语句序列；
 - 综合属性 addr：存放 E 值的地址（名字或临时变量）；
- 全局辅助函数 newTemp()：生成新的临时变量 t，返回 t 的地址（名字）；

```
en(st.get(id.lexeme), "=", E.addr);}
```

赋值语句 SDT:

① $S \rightarrow id = E; \{ S.code = E.code \mid \text{gen}(\text{st.get}(id.\text{lexeme}), "=", E.\text{addr}); \}$

```

    en(st.get(id.lexeme), "=", E.addr);}
    up();
    addr,"=",E1.addr,"+",E2.addr); }

```

- ① $S \rightarrow id = E; \{ S.code = E.code || gen(st.get(id.lexeme), "=", E.addr); \}$
- ② $E \rightarrow E1 + E2 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "+", E2.addr); \}$

```

en(st.get(id.lexeme), "=", E.addr);}
p();
addr,"=",E1.addr,"+",E2.addr); }
p();
addr,"=",E1.addr,"*",E2.addr); }

```

- ① $S \rightarrow id = E; \{ S.code = E.code || gen(st.get(id.lexeme), "=", E.addr); \}$
- ② $E \rightarrow E1 + E2 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "+", E2.addr); \}$
- ③ $E \rightarrow E1 * E2 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "*", E2.addr); \}$

简单算术表达式及赋值语句

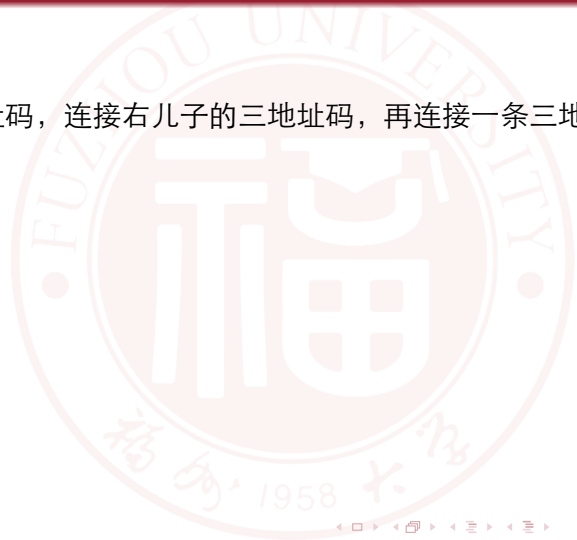
赋值语句 SDT:

- ① $S \rightarrow id = E; \{ S.code = E.code || gen(st.get(id.lexeme), "=", E.addr); \}$
- ② $E \rightarrow E1 + E2 \{ E.addr = newTemp();$
 $E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "+", E2.addr); \}$
- ③ $E \rightarrow E1 * E2 \{ E.addr = newTemp();$
 $E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "*", E2.addr); \}$
- ④ $E \rightarrow -E1 \{ E.addr = newTemp();$
 $E.code = E1.code || E2.code || gen(E.addr, "=", "uminus", E1.addr); \}$

Conclusions

$$C_1 = \frac{1}{2} \left(\frac{1}{2} + \frac{1}{2} \right) = \frac{1}{2}$$

- E 的三地址码为左儿子的三地址码，连接右儿子的三地址码，再连接一条三地址码，



子三地址码序列的连接...

- 子三地址码序列的连接...

子三地址码，连接右儿子的三地址码，再连接一条三地址码序列的连接...

- E 的三地址码为左儿子的三地址码，连接右儿子的三地址码，再连接一条三地址码，
- 其儿子的三地址码为其儿子的儿子三地址码序列的连接...
- ...

增量翻译

- E 的三地址码为左儿子的三地址码，连接右儿子的三地址码，再连接一条三地址码，
- 其儿子的三地址码为其儿子的儿子三地址码序列的连接...
- ...
- 代码序列可能很长，重复使用字符串拼接过程效率低。

增量翻译

- E 的三地址码为左儿子的三地址码，连接右儿子的三地址码，再连接一条三地址码，
- 其儿子的三地址码为其儿子的儿子三地址码序列的连接...
- ...
- 代码序列可能很长，重复使用字符串拼接过程效率低。
- 可以采用**增量**的方式来生成三地址码。

码，连接右儿子的三地址码，再连接一条三地址码的左儿子的三地址码序列的连接...

字符串拼接过程效率低。

三地址码。

造出一个新的三地址指令，还要将它添加到三地址码序列中。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

基于增量翻译的 SDT 方案

- ① $S \rightarrow id = E; \{ \text{gen}(\text{st.get}(id.\text{lexeme}), "=", E.\text{addr}); \}$
- ② $E \rightarrow E1 + E2 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", E1.\text{addr}, "+", E2.\text{addr}); \}$
- ③ $E \rightarrow E1 * E2 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", E1.\text{addr}, "*", E2.\text{addr}); \}$
- ④ $E \rightarrow -E1 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", "uminus", E1.\text{addr}); \}$
- ⑤ $E \rightarrow (E1) \{ E.\text{addr} = E1.\text{addr}(); \}$
- ⑥ $E \rightarrow id \{ E.\text{addr} = \text{st.get}(id.\text{lexeme}); \}$

的基本文法：

| L

地址码时的问题是如何确定数组元素的存放地址

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

例

int a[10];
如何取得 a[i] 的地址?

```
int a[10];
```

如何取得 $a[i]$ 的地址?

例

```
int a[10];
```

如何取得 `a[i]` 的地址?

考虑代码：

```
#include <stdio.h>
int a[100];
int main() {
    printf("%p\n", a);
    printf("a[2].addr=%p\n", &a[2]);
    printf("(void *)a + 2 * 4 = %p\n",
(void *)a + 2 * 4);
    printf("a + 1 = %p\n", a + 1);
    return 0;
}
```

数组元素的寻址

例

C 语言数组声明如下:

```
int a[10];
```

如何取得 $a[i]$ 的地址?

程序输出:

0x100538000

$a[2].addr=0x100538008$

$(void *)a + 2 * 4 = 0x100538008$

$a + 1 = 0x100538004$

考虑代码:

```
#include <stdio.h>
int a[100];
int main() {
    printf("%p\n", a);
    printf("a[2].addr=%p\n", &a[2]);
    printf("(void *)a + 2 * 4 = %p\n",
        (void *)a + 2 * 4);
    printf("a + 1 = %p\n", a + 1);
    return 0;
}
```

- 计算公式:
- $$+ (i - low) * width$$
- $$- low * width + i * width$$

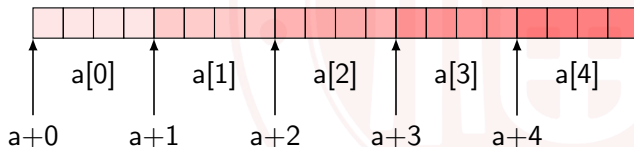
$$base_a + (i - low) * width$$
$$= base_a - low * width + i * width$$

- $$\begin{aligned} & base_a + (i - low) * width \\ = & base_a - \underline{low * width} + i * width^3 \\ = & base_a + i * width \end{aligned}$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

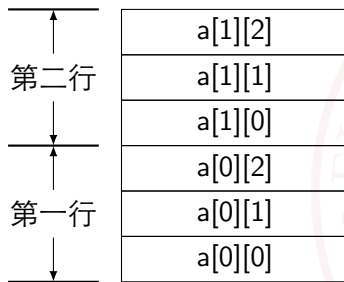
- 一维数组元素 $a[i]$ 的地址的计算公式:

$$\begin{aligned} & base_a + (i - low) * width \\ = & base_a - \underline{low * width} + i * width^3 \\ = & base_a + i * width \end{aligned}$$

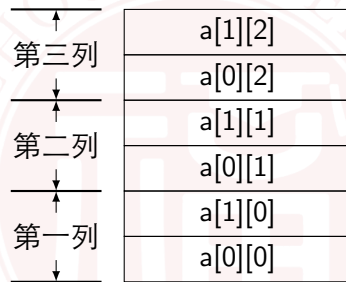


³C 与 Java 等语言数组下标从 0 开始, 即 $low=0$ 。

- 二维数组可以按行或者按列存放：



a[2][3] 按行存放
C/C++/Java



a[2][3] 按列存放
Fortrans/MATLAB⁴

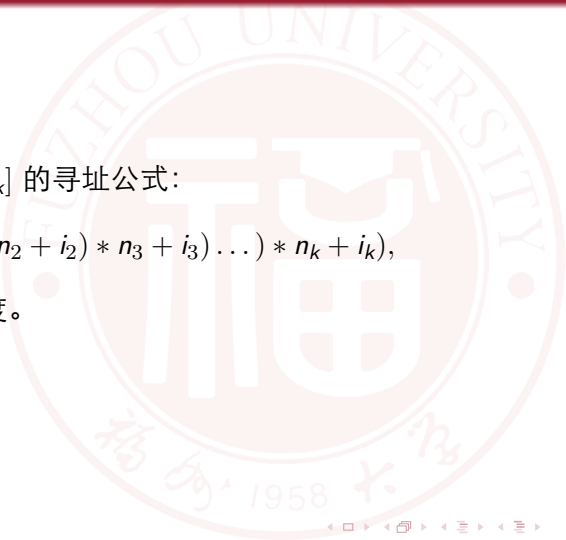
⁴Why?

多位数组的通用公式

k 维行优先数组 a 的元素 $a[i_1, i_2, \dots, i_k]$ 的寻址公式:

$$base_a + e_w * (\dots (i_1 * n_2 + i_2) * n_3 + i_3) \dots) * n_k + i_k),$$

其中, $n_2, n_2, \dots, n_k$ 为对应维度的长度。



数组引用寻址的 SDT

- 数组引用翻译方案的设计中，**关键的问题**是如何将地址计算公式同数组引用的文法关联起来。
- 设非终结符 L 生成一个标识符连接一个下标表达式的序列：

$$L \rightarrow id[E] \mid L_1[E]$$

- L 的综合属性:
 - L.addr 指示一个临时变量, 表示 $\sum_j i_j * w_j$;
 - L.array 是指向数组名字对应符号表条目的指针, 其基地址为 L.array.base;
 - L.type 是 L 生成的子数组的类型。对于任意数组类型 t, 假定 t.width 给出它的宽度, t.elem 给出其元素的类型。

数组引用寻址的 SDT

- $S \rightarrow id = E; \{ \dots \}$
 $| L = E; \{ \text{gen}(L.array.base, '[' L.addr ']', '=', E.addr); \}$
- $E \rightarrow E1 + E2 \mid -E1 \mid (E1) \mid id$
 $| L \{ E.addr = \text{newTemp}(); \text{gen}(E.addr, '=', L.array.base, '[' ,$
 $L.addr, ']'); \}$
- $L \rightarrow id [E] \{ L.array = \text{top.get}(id.lexeme);$
 $L.type = L.array.type.elem; L.addr = \text{newTemp}(); \text{gen}(L.addr, '=',$
 $E.addr, '*', L.type.width); \}$
 $| L1 [E] \{ L.array = L1.array; L.type = L1.type.elem;$
 $t = \text{newTemp}(); L.addr = \text{newTemp}(); \text{gen}(t, '=', E.addr, '*',$
 $L.type.width); \text{gen}(L.addr, '=', L1.addr, '+', t); \}$

例

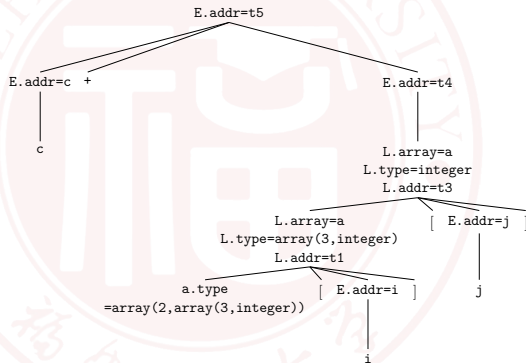
假设有数组 `int a[2][3];`，使用基于数组引用的方案计算表达式 `c+a[i][j]` 的注释分析树及对应的三地址码。

假设有数组 `int a[2][3];`，使用基于数组引用的方案计算表达式 `c+a[i][j]` 的注释分析树及对应的三地址码。

数组 `int a[2][3]`; 和表达式 `c+a[i][j]`

```

S → id=E;      { gen(top.get(id.lexeme), '=', E.addr); }
  | L=E;        { gen(L.array.base, '[' ,L.addr, ']', '=', E.addr); }
E → E1 + E2    { E.addr=newTemp();
                { gen(E.addr, '=', E1.addr, '+', E2.addr); }
  | id          {E.addr=top.get(id.lexeme);}
  | L           {E.addr=newTemp();
                {gen(E.addr, '=', L.array.base, '[' ,L.addr, ']' );}
L → id [ E ]   {L.array=top.get(id.lexeme);
                L.type=L.array.type.elem;
                L.addr=newTemp();
                {gen(L.addr, '=', E.addr, '*', L.type.width);}
  | L1 [ E ]   {L.array=L1.array;
                L.type=L1.type.elem;
                t=newTemp();
                L.addr=newTemp();
                {gen(t, '=', E.addr, '*', L.type.width);
                {gen(L.addr, '=', L1.addr, '+', t);}
    
```



器会：
错）；

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行一些自动的类型转换；

器会：
错)；
高层；

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行一些自动的类型转换；
- 拓宽转换和窄化转换：
 - 拓宽：较低层的类型转换为较高层；
 - 窄化：较高层到较低层。

器会：
错)；
高层；
员写出代码

- 器会：
错)；
高层；
员写出代码



① 知识点回顾

② 中间代码生成策略

声明处理

表达式翻译

控制结构处理

布尔表达式值的计算方法

本式的翻译是结合在一起的。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

式的翻译是结合在一起的。
指令构成的跳转代码序列。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

式的翻译是结合在一起的。
指令构成的跳转代码序列。
尔变量或关系表达式结合在一起。

- 控制流语句的翻译和对布尔表达式的翻译是结合在一起的。
- 布尔表达式 B 被翻译成由跳转指令构成的跳转代码序列。
- 布尔表达式通过布尔运算符将布尔变量或关系表达式结合在一起。

式的翻译是结合在一起的。
指令构成的跳转代码序列。
尔变量或关系表达式结合在一起。

- 式的翻译是结合在一起的。
指令构成的跳转代码序列。
尔变量或关系表达式结合在一起。

式的翻译是结合在一起的。
指令构成的跳转代码序列。
尔变量或关系表达式结合在一起。

- 式的翻译是结合在一起的。
指令构成的跳转代码序列。
尔变量或关系表达式结合在一起。

式的翻译是结合在一起的。

指令构成的跳转代码序列。

布尔变量或关系表达式结合在一起。

if-then-else 和 while-do 等) 中的条件表达式。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻



① 知识点回顾

② 中间代码生成策略

声明处理

表达式翻译

控制结构处理

布尔表达式值的计算方法

$$\begin{aligned} & 1 || (!0 \& \& 0) || 0 \\ &= 1 || (1 \& \& 0) || 0 \\ &= 1 || 0 || 0 \\ &= 1 || 0 \\ &= 1 \end{aligned}$$

数值法的问题

- 会做很多没有意义的运算。
- 遇到大表达式会拖慢速度。

例

```
1  if (a!=b && a!=c && a!=d && a!=e && a!=f && a!=h && a!=i && a!=j
2      && a!=k && b!=c && b!=d && b!=e && b!=f && b!=h && b!=i && b!=j
3      && b!=k && c!=d && c!=e && c!=f && c!=h && c!=i && c!=j && c!=k
4      && d!=e && d!=f && d!=h && d!=i && d!=j && d!=k
5      && e!=f && e!=h && e!=i && e!=j && e!=k
6      && f!=h && f!=i && f!=j && f!=k && i!=j && h!=k)
7  {
8      total++;
9  }
```

优化思路

- 根据布尔表达式的特点：
 - $A \ || \ B$: 若 A 为真, 则 B 的值就不必计算, $A \ || \ B$ 的值为真。
 - $A \ \&\& \ B$: 若 A 为假, 则 B 的值不必计算, $A \ \&\& \ B$ 的值为假。

短路代码

- 在短路（跳转）代码中，布尔运算符 `&&`，`||`，`!` 被翻译成跳转指令。运算符本身不出现在代码中。

例

if (x < 100 || x > 200 && x != y) x = 0; 的三地址码:

if x<100 goto L2

ifFalse x>200 goto L1

ifFalse x!=y goto L1

L2: x=0

L1: nop

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- 考虑如下文法:
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;

- 考虑如下文法:
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;
- 设计继承属性如下:

True 时的跳转;

- 考虑如下文法：
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;
- 设计继承属性如下：
 - B.true: 表达式 B 返回值为 True 时的跳转;

控制流语句的翻译

- 考虑如下文法：
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;
- 设计继承属性如下：
 - B.true: 表达式 B 返回值为 True 时的跳转;
 - B.false: 表达式 B 返回值为 False 时的跳转;



控制流语句的翻译

- 考虑如下文法：
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;
- 设计继承属性如下：
 - B.true: 表达式 B 返回值为 True 时的跳转;
 - B.false: 表达式 B 返回值为 False 时的跳转;
 - S.next: 本语句执行后的跳转点。



控制流语句的翻译

- 考虑如下文法：
 - if (B) then S1;
 - if (B) then S1 else S2;
 - while (B) do S1;
- 设计继承属性如下：
 - B.true: 表达式 B 返回值为 True 时的跳转;
 - B.false: 表达式 B 返回值为 False 时的跳转;
 - S.next: 本语句执行后的跳转点。
- 引入新的辅助函数:



控制流语句的翻译

- 考虑如下文法：
 - `if (B) then S1;`
 - `if (B) then S1 else S2;`
 - `while (B) do S1;`
- 设计继承属性如下：
 - `B.true`: 表达式 B 返回值为 True 时的跳转;
 - `B.false`: 表达式 B 返回值为 False 时的跳转;
 - `S.next`: 本语句执行后的跳转点。
- 引入新的辅助函数：
 - `label = newLabel();` 产生一个新的标签值;

控制流语句的翻译

- 考虑如下文法：
 - `if (B) then S1;`
 - `if (B) then S1 else S2;`
 - `while (B) do S1;`
- 设计继承属性如下：
 - `B.true`: 表达式 `B` 返回值为 `True` 时的跳转;
 - `B.false`: 表达式 `B` 返回值为 `False` 时的跳转;
 - `S.next`: 本语句执行后的跳转点。
- 引入新的辅助函数：
 - `label = newLabel();`: 产生一个新的标签值;
 - `gen(label, ':')`: 给紧随其后的三地址码添加标签。